

importing libraries

```
In [49]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
```

Reading csv

```
In [50]: df= pd.read_csv("e commerce project dataset.csv",encoding= 'latin-1')
```

```
In [51]: df.head()
```

```
Out[51]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer

5 rows × 21 columns

DATA PRE-PROCESSING

```
In [52]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   Row ID                9994 non-null   int64  
 1   Order ID              9994 non-null   object  
 2   Order Date            9994 non-null   object  
 3   Ship Date             9994 non-null   object  
 4   Ship Mode             9994 non-null   object  
 5   Customer ID          9994 non-null   object  
 6   Customer Name         9994 non-null   object  
 7   Segment              9994 non-null   object  
 8   Country              9994 non-null   object  
 9   City                 9994 non-null   object  
10   State                9994 non-null   object  
11   Postal Code          9994 non-null   int64  
12   Region               9994 non-null   object  
13   Product ID           9994 non-null   object  
14   Category             9994 non-null   object  
15   Sub-Category         9994 non-null   object  
16   Product Name         9994 non-null   object  
17   Sales                9994 non-null   float64 
18   Quantity             9994 non-null   int64  
19   Discount             9994 non-null   float64 
20   Profit               9994 non-null   float64 
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

coverting object data type of dates to datetime format

```
In [53]: df['Order Date']=pd.to_datetime(df['Order Date'] , format='mixed', errors='coerce')
df['Ship Date']=pd.to_datetime(df['Ship Date'] , format='mixed', errors='coerce')

df.head()
```

Out[53]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer

5 rows × 21 columns

In [54]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Row ID                9994 non-null  int64  
 1   Order ID              9994 non-null  object  
 2   Order Date            9994 non-null  datetime64[ns]
 3   Ship Date             9994 non-null  datetime64[ns]
 4   Ship Mode             9994 non-null  object  
 5   Customer ID           9994 non-null  object  
 6   Customer Name         9994 non-null  object  
 7   Segment              9994 non-null  object  
 8   Country               9994 non-null  object  
 9   City                 9994 non-null  object  
10   State                9994 non-null  object  
11   Postal Code           9994 non-null  int64  
12   Region               9994 non-null  object  
13   Product ID           9994 non-null  object  
14   Category             9994 non-null  object  
15   Sub-Category         9994 non-null  object  
16   Product Name         9994 non-null  object  
17   Sales                9994 non-null  float64 
18   Quantity             9994 non-null  int64  
19   Discount             9994 non-null  float64 
20   Profit               9994 non-null  float64 
dtypes: datetime64[ns](2), float64(3), int64(3), object(13)
memory usage: 1.6+ MB
```

checking for duplicates

```
In [55]: df.duplicated().sum()
```

```
Out[55]: np.int64(0)
```

```
In [56]: df.columns
```

```
Out[56]: Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
               'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
               'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
               'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
              dtype='object')
```

```
In [57]: df.describe()
```

Out[57]:

	Row ID	Order Date	Ship Date	Postal Code	Sales	
count	9994.000000	9994	9994	9994.000000	9994.000000	999
mean	4997.500000	2016-04-30 00:07:12.259355648	2016-05-03 23:06:58.571142912	55190.379428	229.858001	
min	1.000000	2014-01-03 00:00:00	2014-01-07 00:00:00	1040.000000	0.444000	
25%	2499.250000	2015-05-23 00:00:00	2015-05-27 00:00:00	23223.000000	17.280000	
50%	4997.500000	2016-06-26 00:00:00	2016-06-29 00:00:00	56430.500000	54.490000	
75%	7495.750000	2017-05-14 00:00:00	2017-05-18 00:00:00	90008.000000	209.940000	
max	9994.000000	2017-12-30 00:00:00	2018-01-05 00:00:00	99301.000000	22638.480000	1
std	2885.163629	NaN	NaN	32063.693350	623.245101	

checking for null values

In [58]:

```
df.isna().sum()  
  
# although no null values , as given in info
```

Out[58]:

Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
dtype:	int64

```
In [59]: df['Order Year'] = df['Order Date'].dt.year
df['Order Month'] = df['Order Date'].dt.month
df['Order Day of week'] = df['Order Date'].dt.day_of_week
```

```
In [60]: df.head()
```

Out[60]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer

5 rows × 24 columns

```
In [61]: print(df['Order Year'].dtypes)
print(df['Order Month'].dtypes)
print(df['Order Day of week'].dtypes)
```

```
int32
int32
int32
```

```
In [62]: df.columns
```

```
Out[62]: Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
               'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
               'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
               'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit', 'Order Year',
               'Order Month', 'Order Day of week'],
              dtype='object')
```

```
In [63]: df['Order Month'].describe()
```

```
Out[63]: count      9994.000000
         mean         7.809686
         std         3.284654
         min         1.000000
         25%         5.000000
         50%         9.000000
         75%        11.000000
         max        12.000000
         Name: Order Month, dtype: float64
```

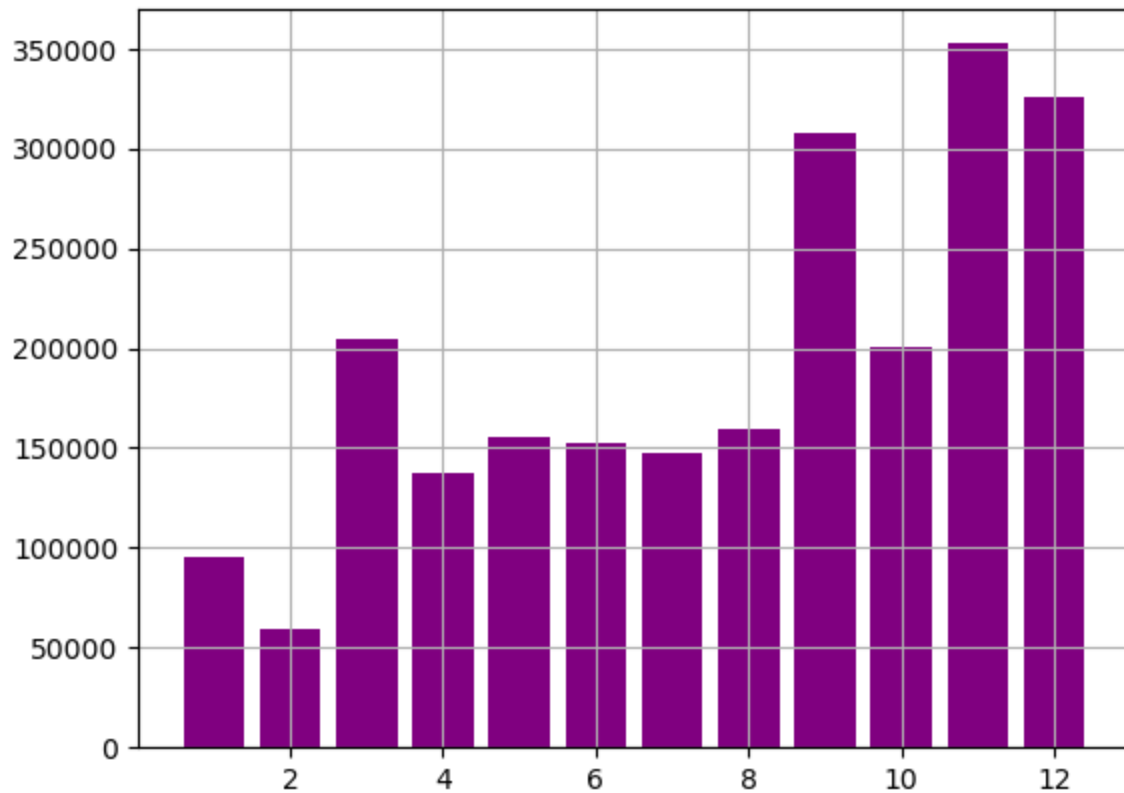
(1) Monthly sales analysis of the store

```
In [64]: sales_by_month= df.groupby('Order Month')['Sales'].sum().sort_values().reset_index(
         sales_by_month
```

```
Out[64]:
```

	Order Month	Sales
0	2	59751.2514
1	1	94924.8356
2	4	137762.1286
3	7	147238.0970
4	6	152718.6793
5	5	155028.8117
6	8	159044.0630
7	10	200322.9847
8	3	205005.4888
9	9	307649.9457
10	12	325293.5035
11	11	352461.0710

```
In [65]: plt.bar(sales_by_month['Order Month'], sales_by_month['Sales'],color = 'purple')
         plt.grid()
         plt.show()
```



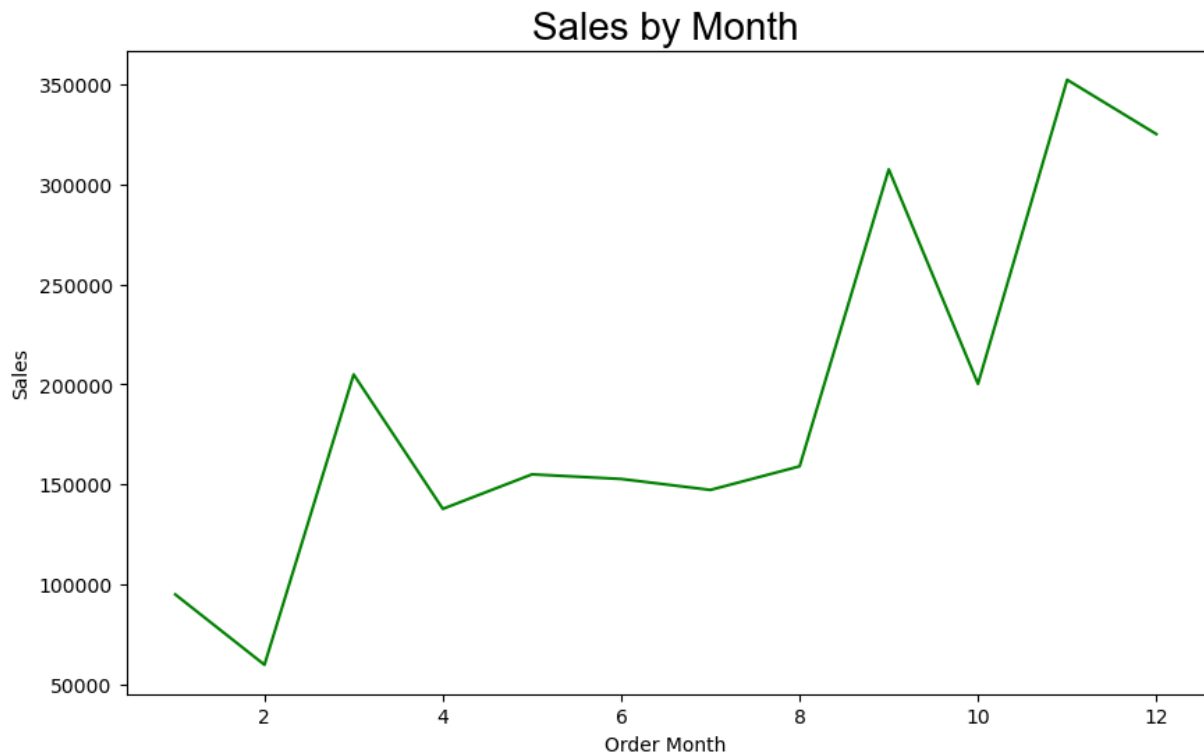
sales were highest in 11th month i.e november month

sales were lowest in the month of february (2nd month)

```
In [66]: sales_by_month= df.groupby('Order Month')['Sales'].sum().reset_index()
```

```
In [67]: plt.figure(figsize= (10,6))
sns.lineplot(
    data = sales_by_month,
    x= sales_by_month['Order Month'],
    y= sales_by_month['Sales'],
    color = 'green'
)

plt.title('Sales by Month', fontsize = '20', fontname ='Arial'
)
plt.show()
```

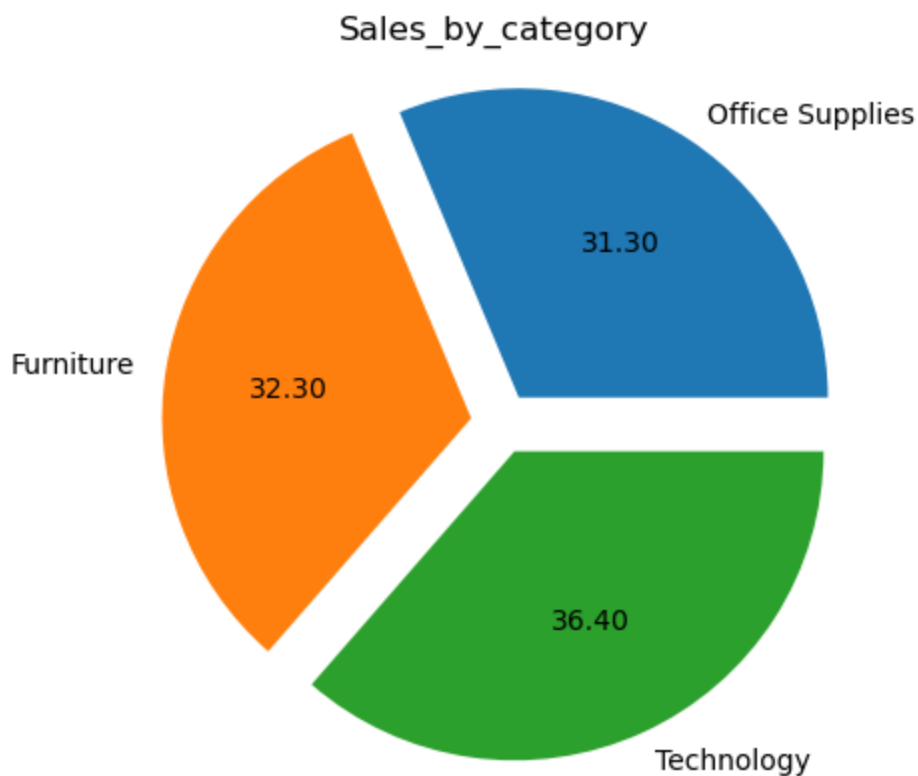
(2) Sales analysis on the basis of product categories

```
In [68]: Sales_by_category = df.groupby('Category')['Sales'].sum().sort_values().reset_index  
Sales_by_category
```

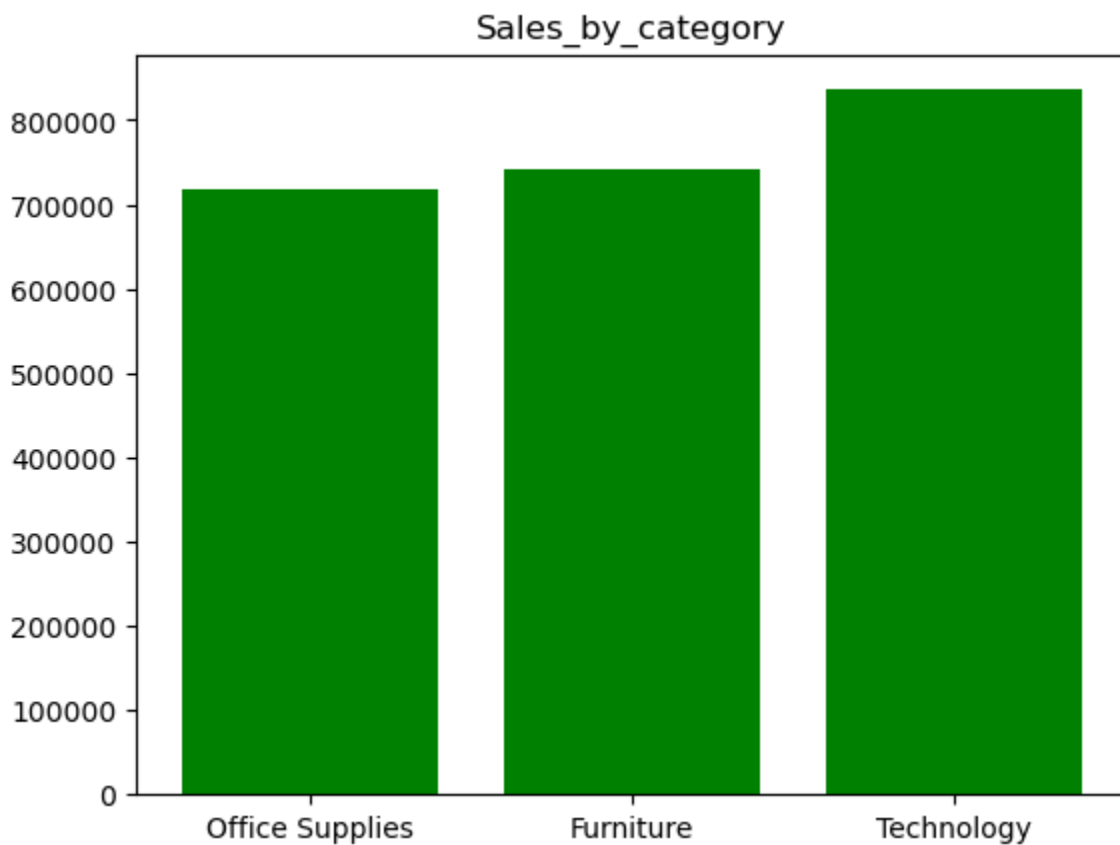
```
Out[68]:
```

	Category	Sales
0	Office Supplies	719047.0320
1	Furniture	741999.7953
2	Technology	836154.0330

```
In [69]: plt.pie(Sales_by_category['Sales'], labels= Sales_by_category['Category'], autopct=  
plt.axis('equal')  
plt.title('Sales_by_category')  
plt.show()
```



```
In [70]: plt.bar(Sales_by_category['Category'], Sales_by_category['Sales'], color='green' )  
plt.title('Sales_by_category')  
plt.show()
```



```
In [71]: Sales_by_category = df.groupby('Category')['Sales'].sum().reset_index()
Sales_by_category
```

```
Out[71]:
```

	Category	Sales
0	Furniture	741999.7953
1	Office Supplies	719047.0320
2	Technology	836154.0330

Technology is the highest selling category and office supplies are the lowest selling category , although all three categories are comparable with each other

(3) Sales analysis on the basis of sub-categories

```
In [72]: df['Sub-Category'].unique()
```

```
Out[72]: array(['Bookcases', 'Chairs', 'Labels', 'Tables', 'Storage',
               'Furnishings', 'Art', 'Phones', 'Binders', 'Appliances', 'Paper',
               'Accessories', 'Envelopes', 'Fasteners', 'Supplies', 'Machines',
               'Copiers'], dtype=object)
```

```
In [73]: df['Sub-Category'].describe()
```

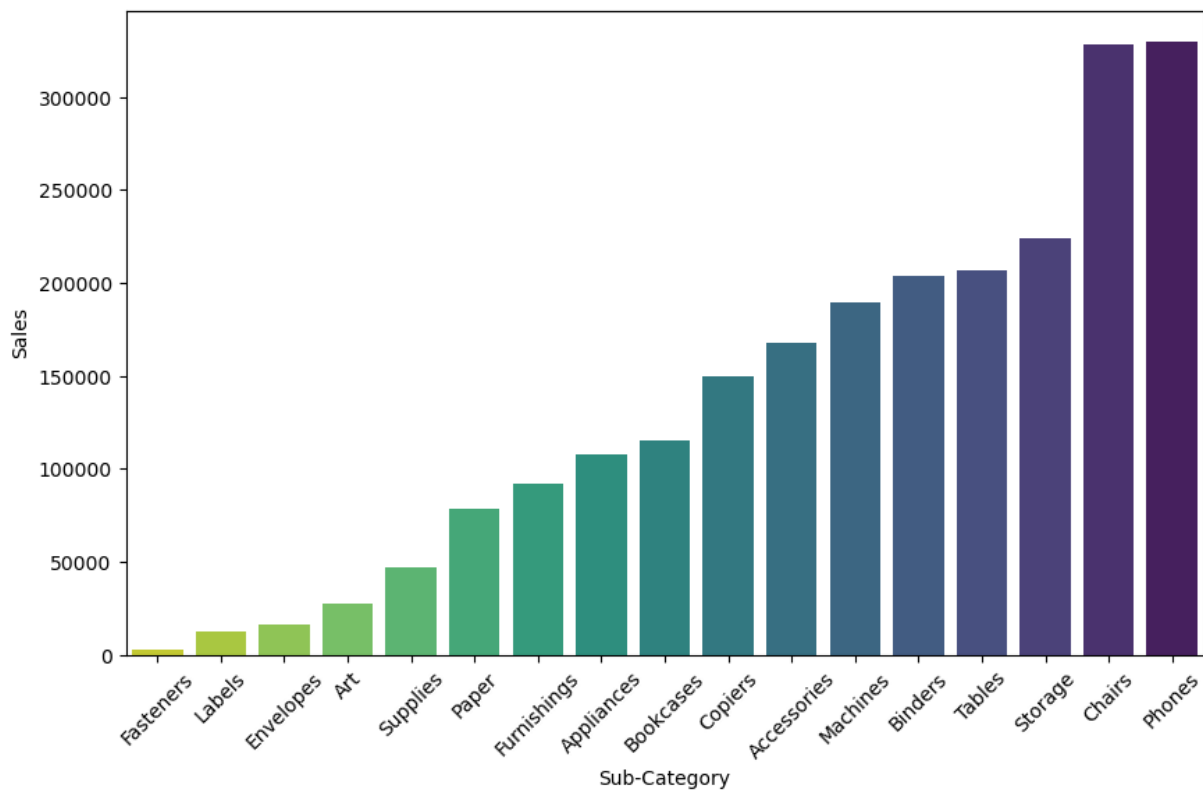
```
Out[73]: count      9994
unique         17
top      Binders
freq         1523
Name: Sub-Category, dtype: object
```

```
In [74]: Sub_Category_sales=df.groupby(['Category', 'Sub-Category'])['Sales'].sum().sort_val
Sub_Category_sales
```

Out[74]:

	Category	Sub-Category	Sales
0	Office Supplies	Fasteners	3024.2800
1	Office Supplies	Labels	12486.3120
2	Office Supplies	Envelopes	16476.4020
3	Office Supplies	Art	27118.7920
4	Office Supplies	Supplies	46673.5380
5	Office Supplies	Paper	78479.2060
6	Furniture	Furnishings	91705.1640
7	Office Supplies	Appliances	107532.1610
8	Furniture	Bookcases	114879.9963
9	Technology	Copiers	149528.0300
10	Technology	Accessories	167380.3180
11	Technology	Machines	189238.6310
12	Office Supplies	Binders	203412.7330
13	Furniture	Tables	206965.5320
14	Office Supplies	Storage	223843.6080
15	Furniture	Chairs	328449.1030
16	Technology	Phones	330007.0540

```
In [75]: plt.figure(figsize= (10,6))
sns.barplot(Sub_Category_sales, x = Sub_Category_sales['Sub-Category'],
            y= Sub_Category_sales['Sales'],
            hue = 'Sub-Category',
            palette= 'viridis_r',
            legend = False)
plt.xticks(rotation=45)
plt.show()
```



Phones(technology category) are the highest selling sub category

Fasteners(office supplies category) are the lowest selling sub category

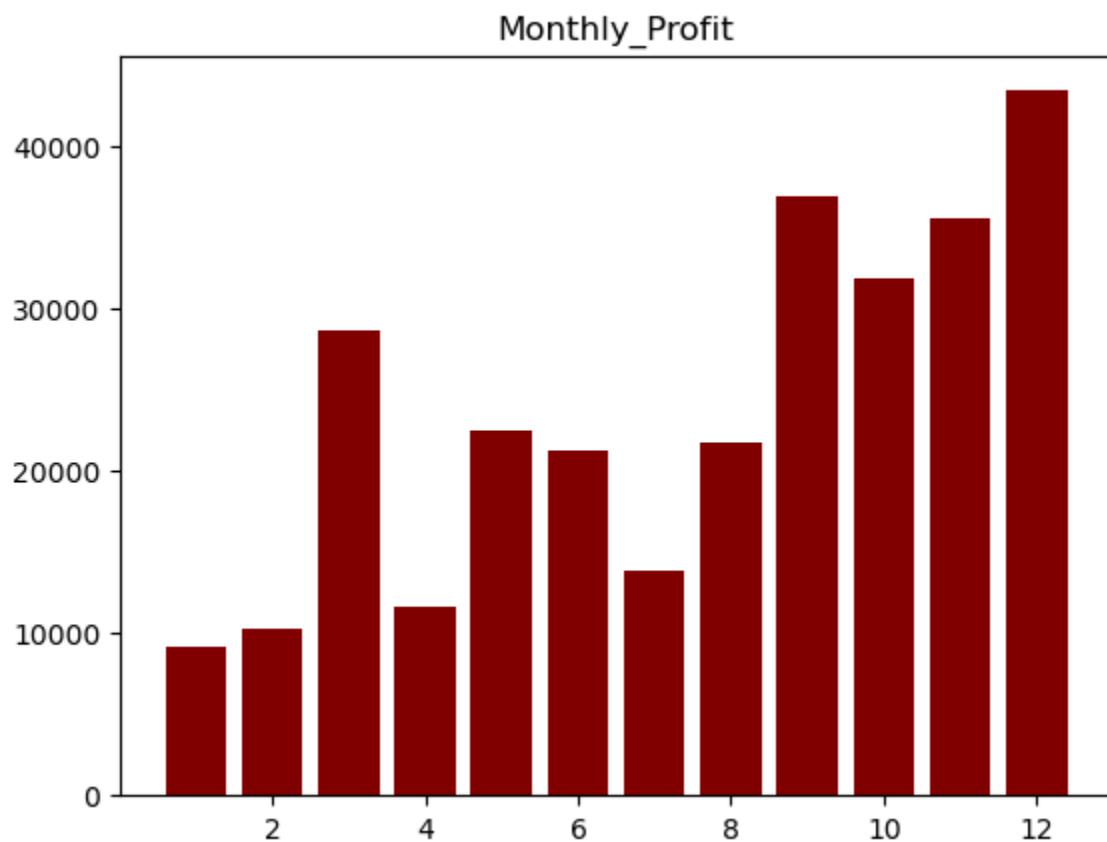
(4) Analysis of monthly profits from sales

```
In [76]: Monthly_Profit = df.groupby('Order Month')['Profit'].sum().sort_values(ascending=False)
Monthly_Profit
```

Out[76]:

	Order Month	Profit
0	12	43369.1919
1	9	36857.4753
2	11	35468.4265
3	10	31784.0413
4	3	28594.6872
5	5	22411.3078
6	8	21776.9384
7	6	21285.7954
8	7	13832.6648
9	4	11587.4363
10	2	10294.6107
11	1	9134.4461

```
In [77]: plt.bar(Monthly_Profit['Order Month'], Monthly_Profit['Profit'],  
               color= 'maroon')  
plt.title('Monthly_Profit')  
plt.show()
```

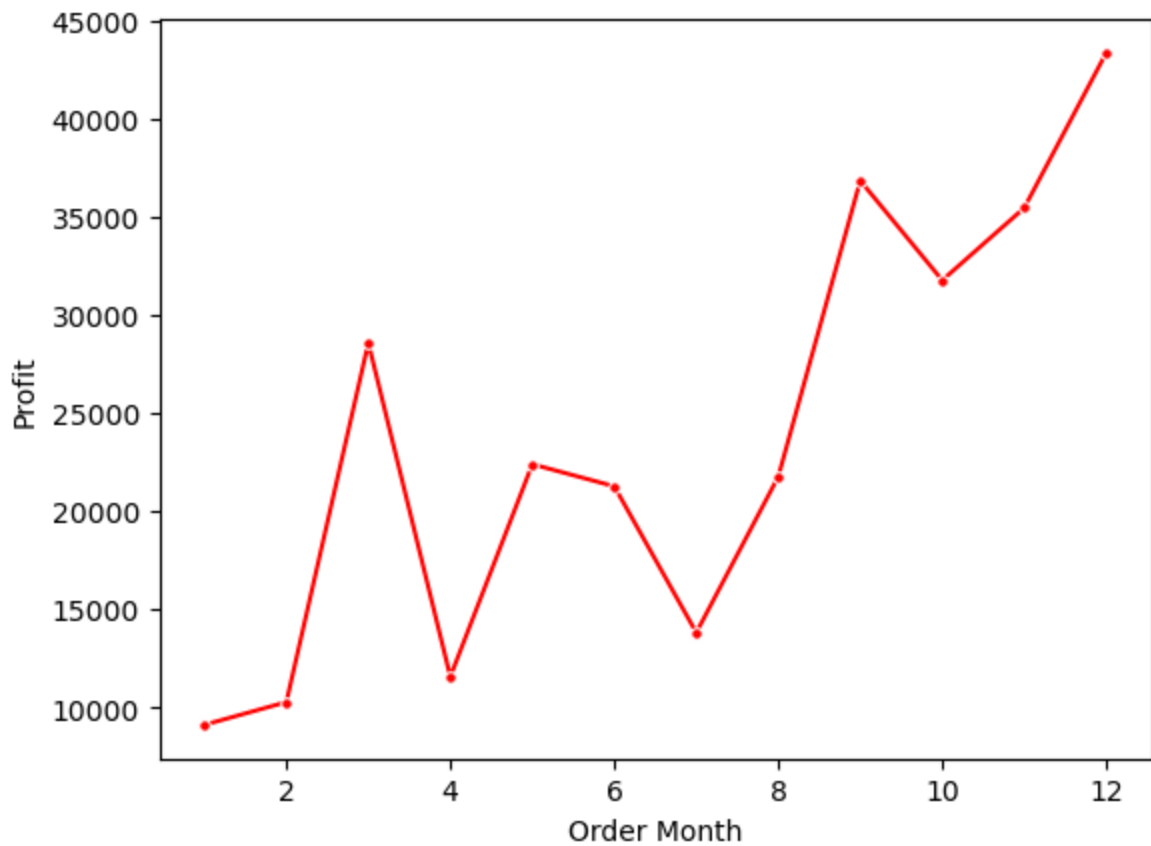


```
In [78]: Monthly_Profit2 = df.groupby('Order Month')['Profit'].sum().reset_index()  
Monthly_Profit2
```

```
Out[78]:
```

	Order Month	Profit
0	1	9134.4461
1	2	10294.6107
2	3	28594.6872
3	4	11587.4363
4	5	22411.3078
5	6	21285.7954
6	7	13832.6648
7	8	21776.9384
8	9	36857.4753
9	10	31784.0413
10	11	35468.4265
11	12	43369.1919

```
In [79]: sns.lineplot(Monthly_Profit2,  
                      x= Monthly_Profit2['Order Month'],  
                      y= Monthly_Profit2['Profit'],  
                      marker = '.',  
                      markersize= 8,  
                      color= 'Red',  
                      legend= False)  
  
plt.show()
```



Highest profit : 12th month - December

Lowest profit : 1st month - January

Profit trend does not perfectly mirror sales — margins vary by month

(5) Analysis of profit by category and sub-category

```
In [80]: Profit_by_category = df.groupby('Category')['Profit'].sum().sort_values().reset_index()
Profit_by_category
```

```
Out[80]:
```

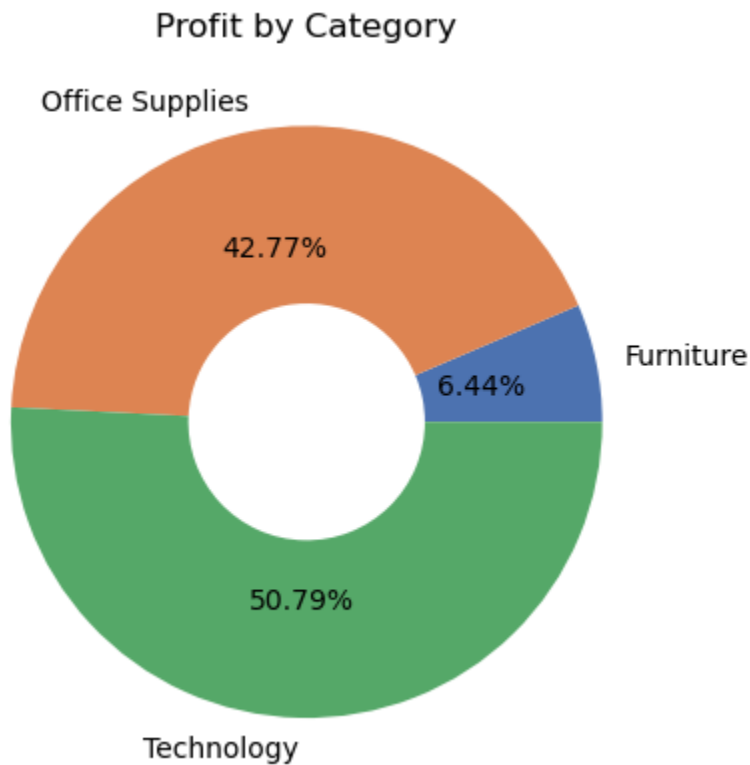
	Category	Profit
0	Furniture	18451.2728
1	Office Supplies	122490.8008
2	Technology	145454.9481

```
In [81]: colors = sns.color_palette('deep')
```



```
plt.pie(Profit_by_category['Profit'],
        labels= Profit_by_category['Category'],
        colors= colors,
        autopct= '%1.2f%%',
        wedgeprops= {'width':0.6}
    )
plt.title('Profit by Category')

plt.show()
```



Highest profit making category - Technology

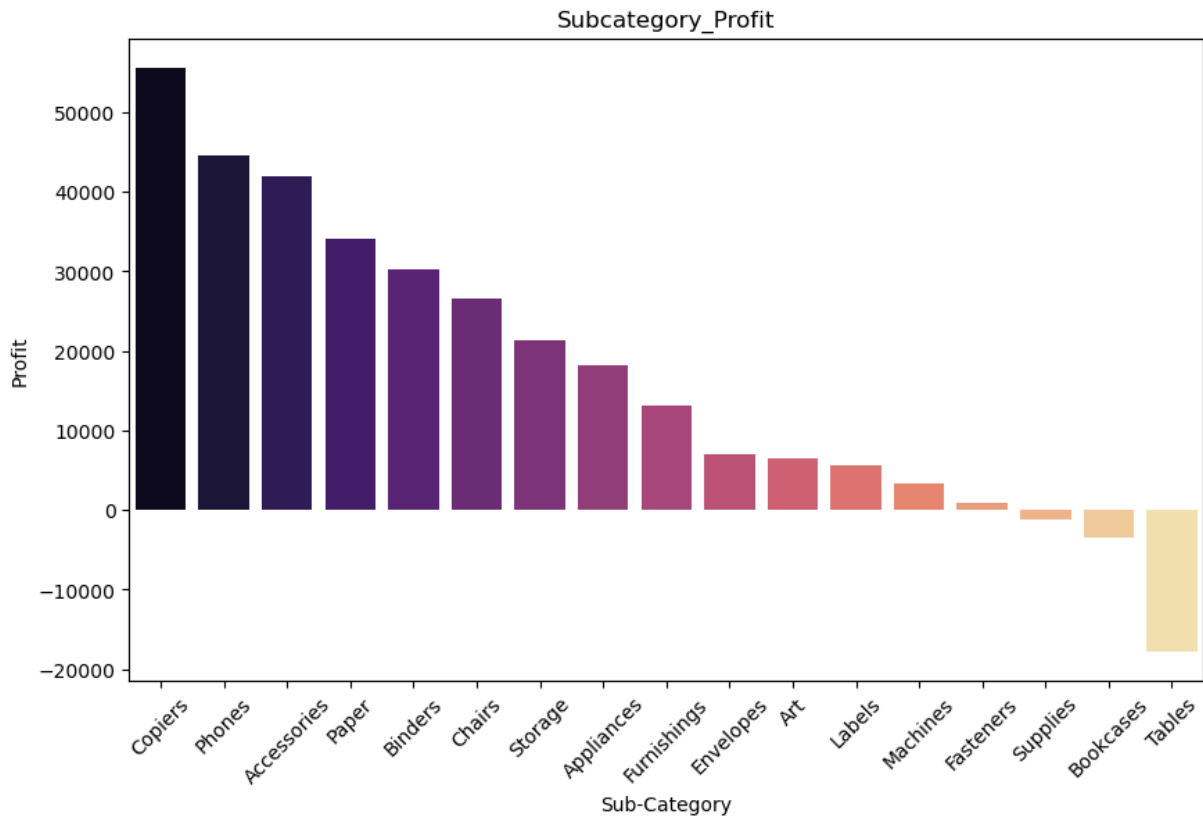
Lowest profit making category - Furniture

```
In [82]: Subcategory_Profit = df.groupby('Sub-Category')['Profit'].sum().sort_values(ascending=True)
Subcategory_Profit
```

Out[82]:

	Sub-Category	Profit
0	Copiers	55617.8249
1	Phones	44515.7306
2	Accessories	41936.6357
3	Paper	34053.5693
4	Binders	30221.7633
5	Chairs	26590.1663
6	Storage	21278.8264
7	Appliances	18138.0054
8	Furnishings	13059.1436
9	Envelopes	6964.1767
10	Art	6527.7870
11	Labels	5546.2540
12	Machines	3384.7569
13	Fasteners	949.5182
14	Supplies	-1189.0995
15	Bookcases	-3472.5560
16	Tables	-17725.4811

```
In [83]: plt.figure(figsize= (10,6))
sns.barplot(Subcategory_Profit,
            x= Subcategory_Profit['Sub-Category'],
            y=Subcategory_Profit['Profit'] ,
            hue = 'Sub-Category',
            palette= 'magma',
            legend= False
            )
plt.xticks(rotation = 45)
plt.title('Subcategory_Profit')
plt.show()
```



```
In [84]: df.groupby('Category')['Sub-Category'].unique().reset_index()
```

```
Out[84]:
```

	Category	Sub-Category
0	Furniture	[Bookcases, Chairs, Tables, Furnishings]
1	Office Supplies	[Labels, Storage, Art, Binders, Appliances, Pa...]
2	Technology	[Phones, Accessories, Machines, Copiers]

Highest profit making sub category -
Copiers(Technology)

Lowest profit making sub category - Tables(Furniture)
- loss making

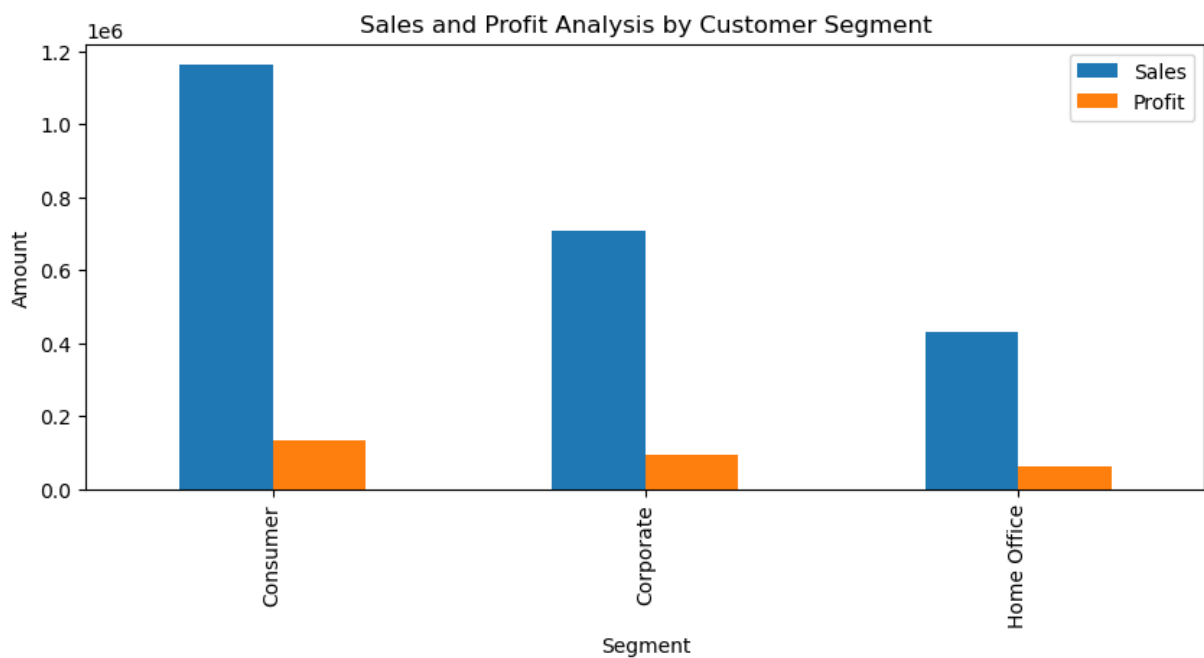
(6) Analysis of sales and profit by customer segment

```
In [85]: Customer_segment= df.groupby('Segment').agg({'Sales':'sum', 'Profit':'sum'}).reset_
Customer_segment
```

```
Out[85]:
```

	Segment	Sales	Profit
0	Consumer	1.161401e+06	134119.2092
1	Corporate	7.061464e+05	91979.1340
2	Home Office	4.296531e+05	60298.6785

```
In [86]: Customer_segment.set_index('Segment')[['Sales', 'Profit']].plot(  
        kind='bar',  
        figsize=(10,4)  
        )  
  
plt.title('Sales and Profit Analysis by Customer Segment')  
plt.ylabel('Amount')  
plt.show()
```



Sales :

Highest - Consumer

Lowest - Home Office

Profit:

Highest - Consumer

Lowest - Home Office

(7) Analysis of sales to profit ratio

```
In [87]: Sales_Profit_by_Segment= df.groupby('Segment').agg({'Sales':'sum', 'Profit':'sum'})
Sales_Profit_by_Segment
```

```
Out[87]:
```

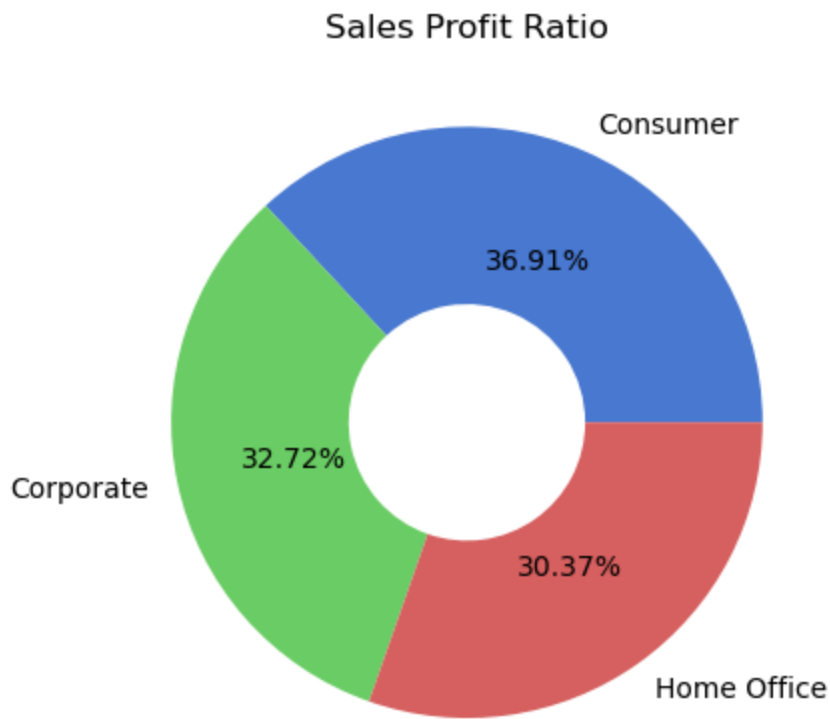
	Segment	Sales	Profit
0	Consumer	1.161401e+06	134119.2092
1	Corporate	7.061464e+05	91979.1340
2	Home Office	4.296531e+05	60298.6785

```
In [88]: Sales_Profit_by_Segment['Ratio']= Sales_Profit_by_Segment['Sales']/Sales_Profit_by_
Sales_Profit_by_Segment
```

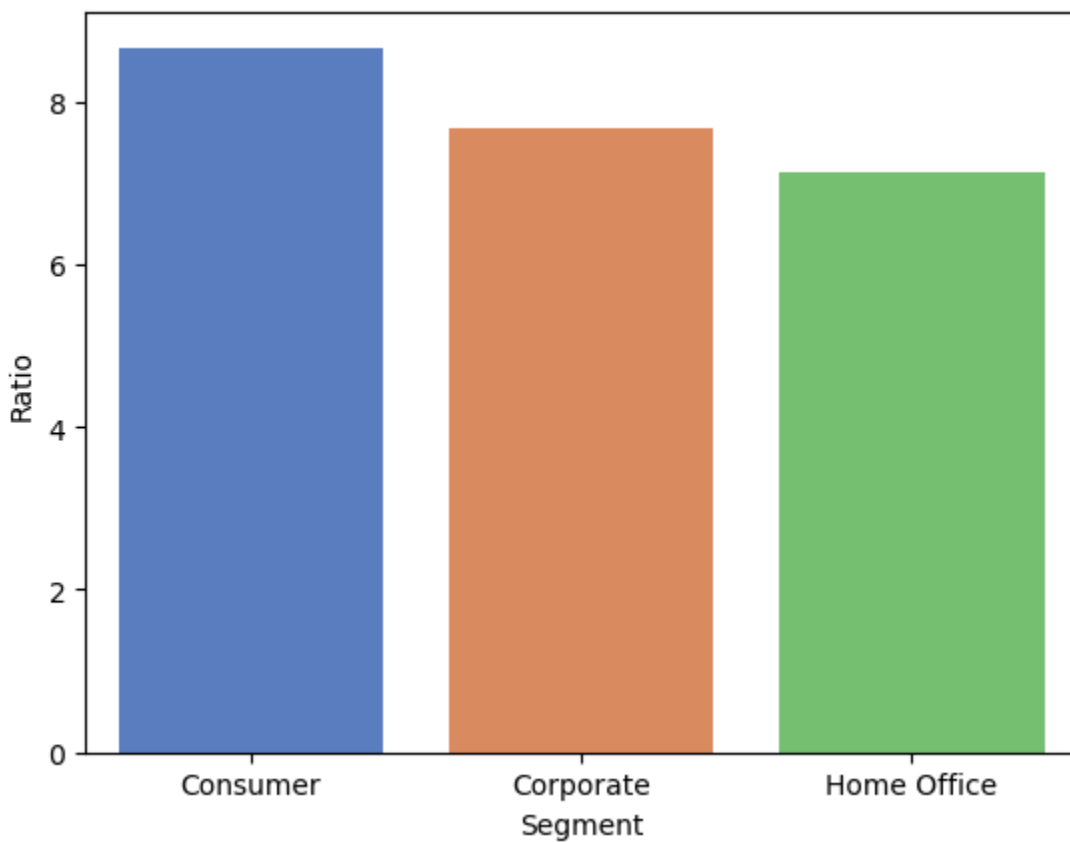
```
Out[88]:
```

	Segment	Sales	Profit	Ratio
0	Consumer	1.161401e+06	134119.2092	8.659471
1	Corporate	7.061464e+05	91979.1340	7.677245
2	Home Office	4.296531e+05	60298.6785	7.125416

```
In [89]: colors = sns.color_palette('muted6')
plt.pie(Sales_Profit_by_Segment['Ratio'] ,
        labels= Sales_Profit_by_Segment['Segment'],
        autopct= "%1.2f%%",
        colors= colors,
        wedgeprops= {'width':0.6}
        )
plt.title('Sales Profit Ratio')
plt.show()
```



```
In [90]: sns.barplot(data= Sales_Profit_by_Segment, x = 'Segment', y = 'Ratio', palette= 'mu
plt.show()
```



HIGHEST : Consumer : 8.659471

LOWEST : Home Office : 7.125416

```
In [91]: df.groupby('State')['Profit'].mean().sort_values(ascending=False).reset_index()
```

Out[91]:

	State	Profit
0	Vermont	204.088936
1	Rhode Island	130.100523
2	Indiana	123.375411
3	Montana	122.221900
4	Minnesota	121.608847
5	District of Columbia	105.958930
6	Delaware	103.930988
7	Wyoming	100.196000
8	Missouri	97.518341
9	Michigan	95.934069
10	Alabama	94.865989
11	Georgia	88.315453
12	Nevada	85.045279
13	Virginia	83.026564
14	Kentucky	80.573357
15	Wisconsin	76.380004
16	New Jersey	75.176260
17	Oklahoma	73.544788
18	Maryland	66.963608
19	Arkansas	66.811452
20	Washington	66.013146
21	New York	65.637011
22	New Hampshire	63.203807
23	Mississippi	59.867475
24	Maine	56.810775
25	Nebraska	53.607742
26	Louisiana	52.288150
27	Massachusetts	50.262975
28	Utah	48.047802
29	West Virginia	46.480400

	State	Profit
30	Connecticut	42.823071
31	South Carolina	42.120395
32	Iowa	39.460397
33	Idaho	39.367767
34	California	38.171608
35	Kansas	34.851813
36	South Dakota	32.902358
37	North Dakota	32.878529
38	New Mexico	31.273408
39	Florida	-8.875461
40	Oregon	-9.600569
41	Arizona	-15.303235
42	Illinois	-25.625787
43	Texas	-26.121174
44	Pennsylvania	-26.507598
45	Tennessee	-29.189583
46	North Carolina	-30.083985
47	Colorado	-35.867351
48	Ohio	-36.186304

(8) Top 10 states with highest Profit

```
In [92]: profit_by_states = df.groupby('State')['Profit'].mean().sort_values(ascending=False)
```

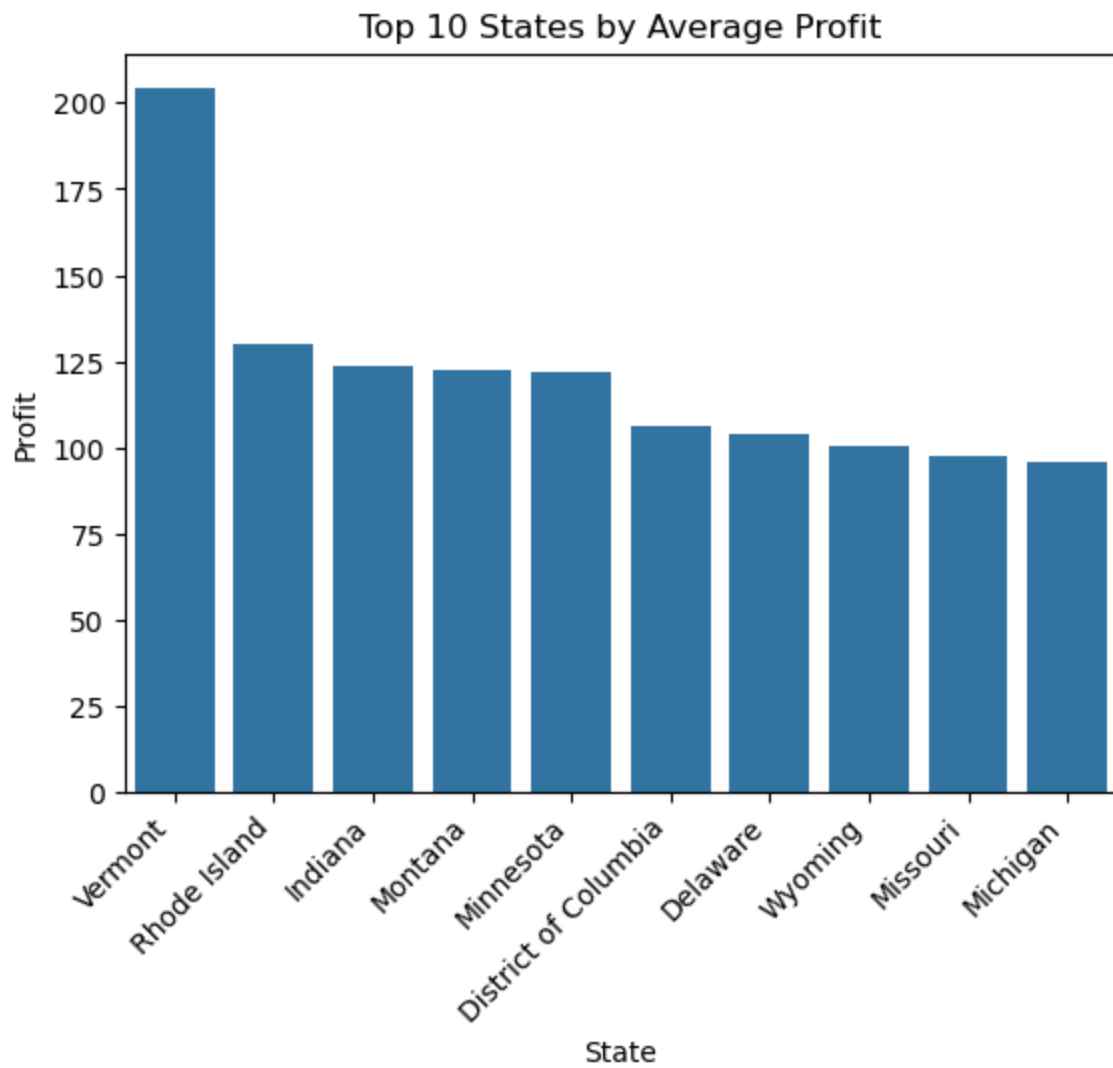
```
In [93]: profit_by_states
```

Out[93]:

	State	Profit
0	Vermont	204.088936
1	Rhode Island	130.100523
2	Indiana	123.375411
3	Montana	122.221900
4	Minnesota	121.608847
5	District of Columbia	105.958930
6	Delaware	103.930988
7	Wyoming	100.196000
8	Missouri	97.518341
9	Michigan	95.934069

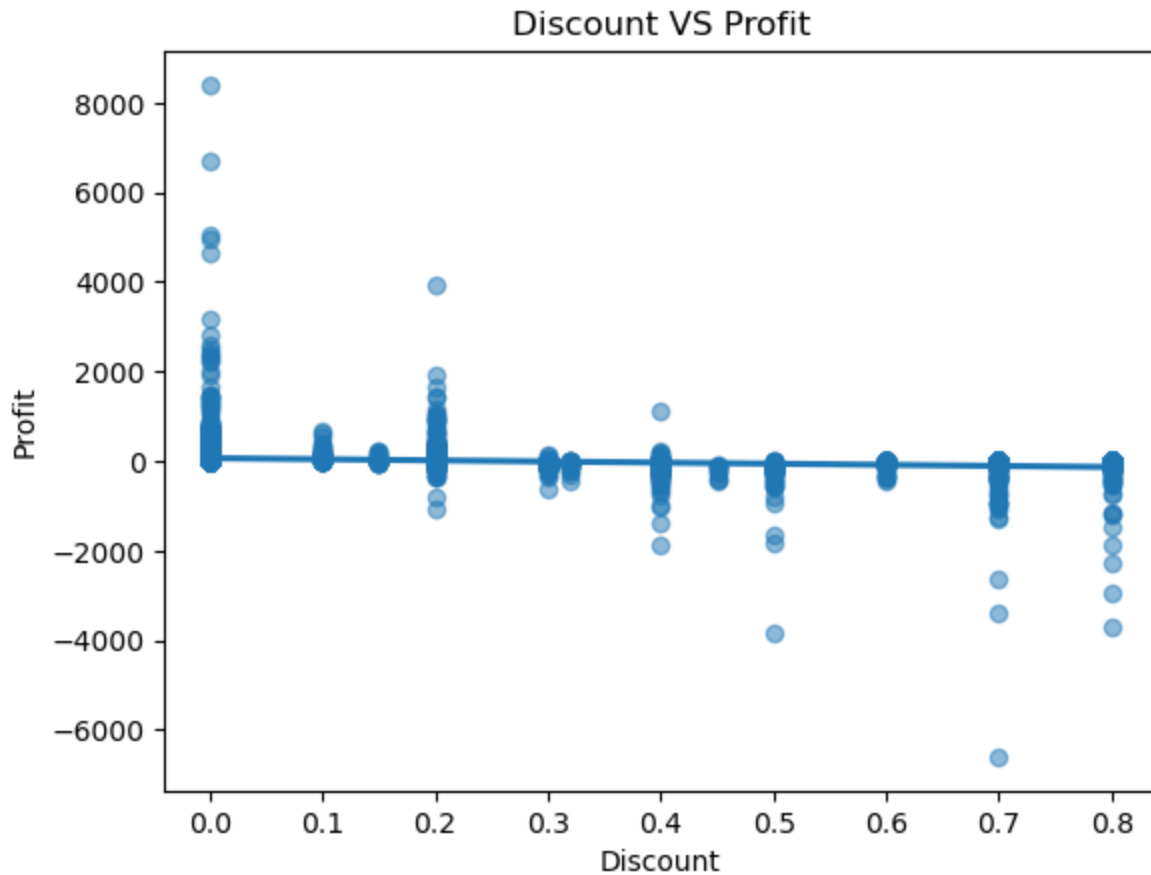
```
In [94]: sns.barplot(data = profit_by_states, x= 'State' , y= 'Profit' )
plt.xticks(rotation = 45, ha = 'right')
# ha -> horizontal alignment , Align the right end of each label with the tick mark
plt.title('Top 10 States by Average Profit')

plt.show()
```



(9) Discount VS Profit

```
In [95]: sns.regplot(data = df , x = 'Discount', y = 'Profit' , scatter_kws={'alpha':0.5})  
plt.title('Discount VS Profit')  
plt.show()
```



Orders with higher discounts tend to generate lower profits, indicating that aggressive discounting can negatively impact profitability.

SUMMARY

(1) Monthly sales analysis of the store

Ans

11th month , ie november had the **highest sales** of Rs.352461.0710

2nd month , ie february had the **lowest sales** of Rs.59751.2514

(2) Sales Analysis on the basis of product categories

Ans

LOWEST SALES => Office supplies

HIGHEST SALES => Technology

(3) Sales analysis on the basis of sub-categories

Ans

HIGHEST SALES : Phones of Technology category

LOWEST SALES : Fasteners of Office supplies category

(4) Analysis of monthly profits from sales

Ans

HIGHEST PROFIT : December (12th month)

LOWEST PROFIT : January (1st month)

Profit trend does not perfectly mirror sales — margins vary by month

(5) Analysis of profit by category and sub-category

Ans

CATEGORY :

HIGHEST PROFIT : Technology

LOWEST PROFIT : Furniture

SUB-CATEGORY :

HIGHEST PROFIT : Copiers of technology category

LOWEST PROFIT : Tables of furniture category

(6) Analysis of sales and profit by customer segment

Ans

SALES :

HIGHEST : Consumer

LOWEST : Home Office

PROFIT :

HIGHEST : Consumer

LOWEST : Home Office

(7) Analysis of sales to profit ratio

Ans

HIGHEST : Consumer : 8.659471

LOWEST : Home Office : 7.125416

(8) Top 10 states with highest Profits

```
In [96]: profit_by_states
```

Out[96]:

	State	Profit
0	Vermont	204.088936
1	Rhode Island	130.100523
2	Indiana	123.375411
3	Montana	122.221900
4	Minnesota	121.608847
5	District of Columbia	105.958930
6	Delaware	103.930988
7	Wyoming	100.196000
8	Missouri	97.518341
9	Michigan	95.934069

(9) Discount VS Profit

Orders with higher discounts tend to generate lower profits, indicating that aggressive discounting can negatively impact profitability.